

Vertex AI：使用 TensorFlow 進行多工作人員訓練和移轉學習

來源：https://codelabs.developers.google.com/vertex_multiworker_training?hl=zh-tw

步驟數：8

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [用途總覽](#)
4. [設定環境](#)
5. [將訓練應用程式程式碼容器化](#)
6. [在 Vertex AI 中執行多工作人員訓練工作](#)
7. [\[選用\] 使用 Vertex SDK](#)
8. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將使用 [Vertex AI](#) 執行 TensorFlow 模型的多工作人員訓練工作。

課程內容

內容如下：

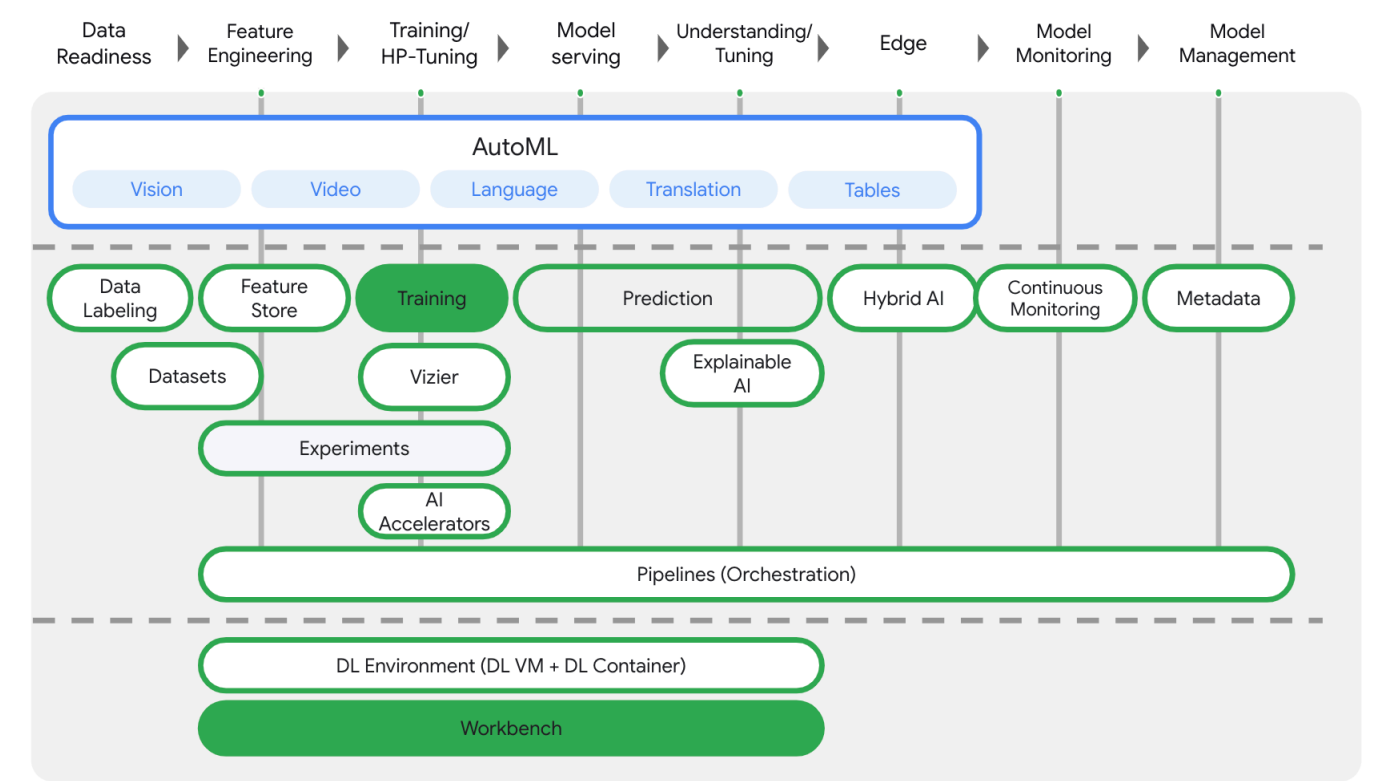
- 修改訓練應用程式程式碼，進行多工作站訓練
- 透過 Vertex AI 使用者介面設定及啟動多工作人員訓練工作
- 使用 Vertex SDK 設定及啟動多工作站訓練工作

在 Google Cloud 上執行這個實驗室的總費用約為 **\$5 美元**。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。[Vertex AI](#) 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。如有任何意見，請參閱[支援頁面](#)。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於下列產品：訓練和 **Workbench**



步驟 3 · 約 5 分鐘

3. 用途總覽

在本實驗室中，您將使用遷移學習，在 [TensorFlow Datasets](#) 的 [木薯資料集](#) 上訓練圖像分類模型。您將使用的架構是 `tf.keras.applications` 程式庫中的 ResNet50 模型，該模型已在 [Imagenet 資料集](#) 上預先訓練。

為什麼要使用分散式訓練？

如果您只有一個 GPU，TensorFlow 會使用這個加速器加快模型訓練速度，您不需要額外進行任何作業。不過，如要透過單一機器或多部機器 (每部機器可能有多個 GPU) 使用多個 GPU，進一步提升效能，則必須使用 `tf.distribute`，這是 TensorFlow 的程式庫，可跨多個裝置執行運算。裝置是指 CPU 或加速器，例如 GPU 或 TPU，TensorFlow 可在某些機器上執行運算。

如要開始使用分散式訓練，最簡單的方法就是使用配備多部 GPU 裝置的單一機器。`tf.distribute` 模組的 TensorFlow 分散式策略會管理所有 GPU 的資料分配和梯度更新協調作業。如果您已精通單一主機訓練，並希望進一步擴大規模，那麼在叢集中新增多部機器，有助於大幅提升效能。您可以運用僅含 CPU 的機器叢集，或每部機器都有一或多個 GPU 的叢集。本實驗室將說明後者，並示範如何使用 `MultiWorkerMirroredStrategy`，在 Vertex AI 的多部機器上分散訓練 TensorFlow 模型。

`MultiWorkerMirroredStrategy` 是一種同步資料平行化策略，只需變更幾行程式碼即可使用。系統會在叢集中的每部裝置上建立模型副本。後續的梯度更新會以同步方式進行。也就是說，每個 worker 裝置會針對不同的輸入資料部分，透過模型計算正向和反向傳遞。接著，系統會彙整機器上所有裝置和叢集中所有機器的這些切片，並在稱為「全歸約」的程式中縮減 (通常是平均) 這些切片。然後，最佳化工具會使用這些縮減的梯度執行參數更新，讓裝置保持同步。如要進一步瞭解如何使用 TensorFlow 進行分散式訓練，請觀看下方影片：



步驟 4 · 約 10 分鐘

4. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

步驟 1：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。您需要這項資訊才能建立筆記本執行個體。

步驟 2：啟用 Container Registry API

前往「Container Registry」，然後選取「啟用」（如果尚未啟用）。您將使用這個工具為自訂訓練工作建立容器。

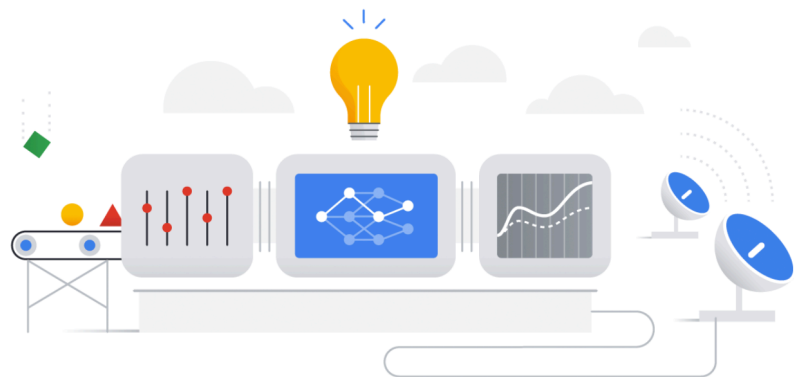
步驟 3：啟用 Vertex AI API

前往 [Cloud 控制台](#) 的 [Vertex AI 專區](#)，然後點選「啟用 Vertex AI API」。

Get started with Vertex AI

Vertex AI empowers machine learning developers, data scientists, and data engineers to take their projects from ideation to deployment, quickly and cost-effectively. [Learn more](#)

ENABLE VERTEX AI API



Region
us-central1 (Iowa)

Prepare your training data

Collect and prepare your data, then import it into a dataset to train a model

+ CREATE DATASET

Train your model

Train a best-in-class machine learning model with your dataset. Use Google's AutoML, or bring your own code.

+ TRAIN NEW MODEL

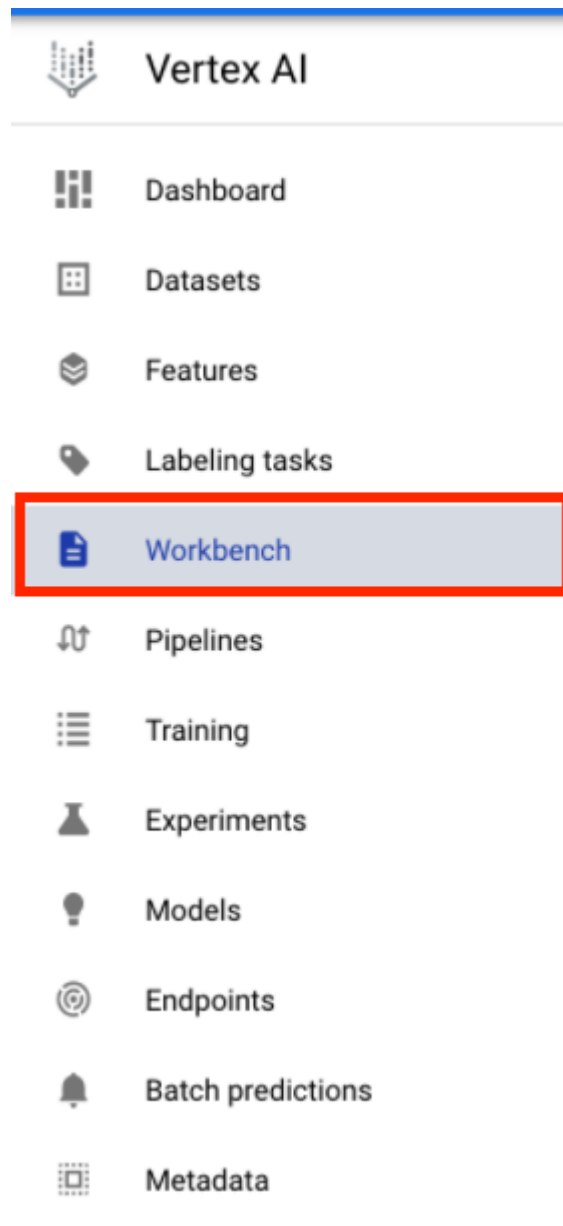
Get predictions

After you train a model, you can use it to get predictions, either online as an endpoint or through batch requests

+ CREATE BATCH PREDICTION

步驟 4：建立 Vertex AI Workbench 執行個體

在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



如果尚未啟用 Notebooks API，請先啟用。



Notebooks API

Google Enterprise API

Notebooks API is used to manage notebook resources in Google Cloud.

ENABLE

TRY THIS API [↗](#)

OVERVIEW

PRICING

DOCUMENTATION

Overview

Notebooks API is used to manage notebook resources in Google Cloud.

Additional details

Type: [SaaS & APIs](#)

Last updated: 15/09/2021

Category: [Google Enterprise APIs](#)

Service name: notebooks.googleapis.com

啟用後，按一下「MANAGED NOTEBOOKS」(代管型筆記本)：

Managed notebooks

[+ NEW NOTEBOOK](#)

[REFRESH](#)

[▶ START](#)

[■ STOP](#)

[⏻ RESET](#)

[🗑 DELETE](#)

[MANAGED NOTEBOOKS](#) [PREVIEW](#)

USER-MANAGED NOTEBOOKS

EXECUTIONS [PREVIEW](#)

SCHEDULES [PREVIEW](#)

Managed notebooks provide JupyterLab services and flexible computing resources integrated with Google Cloud services. [Learn more](#)

然後選取「新增記事本」。

[+ NEW NOTEBOOK](#)

[REFRESH](#)

[▶ START](#)

[MANAGED NOTEBOOKS](#) [PREVIEW](#)

USER-MANAGED NOTEBOOKS

Notebooks service has been moved under the Vertex AI Workbench

為筆記本命名，然後按一下「進階設定」。

Create a managed notebook

Notebook name *

my-notebook

Name must be 63 characters or fewer, must start with a letter and include only lowercase letters, digits or '-'.
Name must be 63 characters or fewer, must start with a letter and include only lowercase letters, digits or '-'.

Region *

us-central1 (Iowa)



Advanced settings



在「進階設定」下方啟用閒置關機功能，並將分鐘數設為 60。也就是說，筆記本會在閒置時自動關機，避免產生不必要的費用。

Idle shutdown

☒ Enable Idle Shutdown 

Time of inactivity before shutdown (Minutes) *

60

Must be integer: 1-600

在「安全性」下方，選取「啟用終端機」(如果尚未啟用)。

Security

☒ Enable nbconvert 

☒ Enable file downloading from Notebook UI

☒ Enable terminal 

其他進階設定可以保留原樣。

接著點選「建立」。執行個體會在幾分鐘內佈建完畢。

建立執行個體後，請選取「Open JupyterLab」。



my-notebook

OPEN JUPYTERLAB

首次使用新執行個體時，系統會要求您驗證身分。按照 UI 中的步驟操作。

Authenticate your managed notebook

To use managed notebooks with your Google Cloud Platform data, you need to grant the Google Cloud SDK permission to access your data and authenticate your managed notebook.

Click the button below to get your authentication code.

Get authentication code 

EXIT

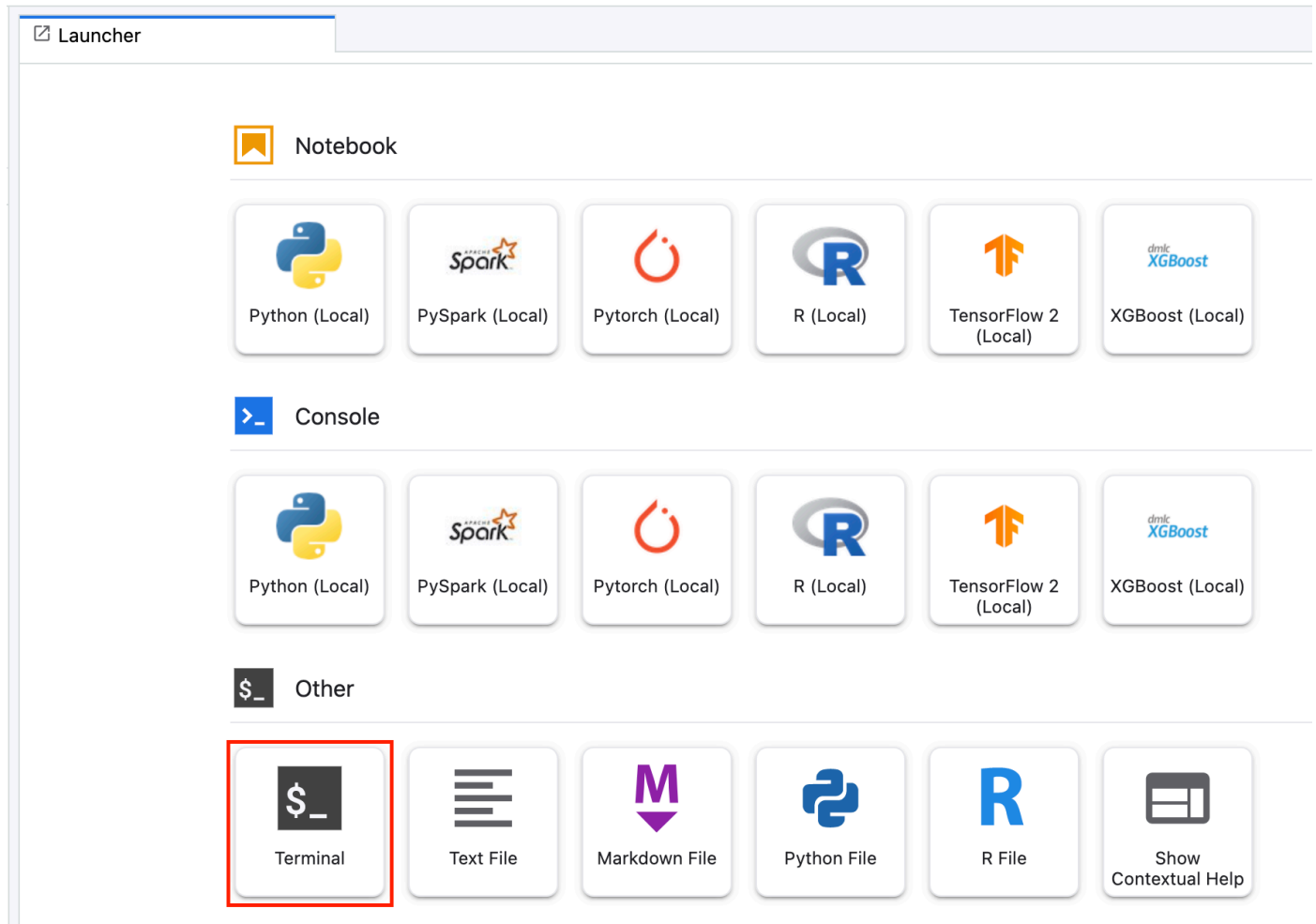
AUTHENTICATE

步驟 5 · 約 10 分鐘

5. 將訓練應用程式程式碼容器化

您要將訓練應用程式程式碼放入 [Docker 容器](#)，並將這個容器推送至 [Google Container Registry](#)，藉此將訓練工作提交至 Vertex。使用這種方法，您可以訓練以任何架構建構的模型。

首先，請從 Launcher 選單開啟筆記本執行個體中的終端機視窗：



建立名為 `cassava` 的新目錄，然後切換至該目錄：

```
mkdir cassava
cd cassava
```

步驟 1：建立 Dockerfile

將程式碼容器化的第一步是建立 Dockerfile。Dockerfile 包含執行映像檔所需的所有指令。這會安裝所有必要的程式庫，並設定訓練程式碼的進入點。

在終端機中建立空白的 Dockerfile：

```
touch Dockerfile
```

開啟 Dockerfile，然後將下列內容複製到其中：

```
FROM gcr.io/deeplearning-platform-release/tf2-gpu.2-7

WORKDIR /

# Copies the trainer code to the docker image.
COPY trainer /trainer

# Sets up the entry point to invoke the trainer.
ENTRYPOINT ["python", "-m", "trainer.task"]
```

這個 Dockerfile 使用 [深度學習容器 TensorFlow 企業版 2.7 GPU Docker 映像檔](#)。Google Cloud 上的深度學習容器預先安裝了許多常見的機器學習和資料科學框架。下載該映像檔後，這個 Dockerfile 會設定訓練程式碼的進入點。您尚未建立這些檔案，在下一個步驟中，您將新增用於訓練及調整模型的程式碼。

步驟 2：建立 Cloud Storage 值區

在這項訓練工作中，您會將經過訓練的 TensorFlow 模型匯出至 Cloud Storage Bucket。在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

接著，在終端機中執行下列指令，在專案中建立新的 bucket。

```
BUCKET="gs://${PROJECT_ID}-bucket"
gsutil mb -l us-central1 $BUCKET
```

步驟 3：新增模型訓練程式碼

在終端機中執行下列指令，為訓練程式碼建立目錄，以及新增程式碼的 Python 檔案：

```
mkdir trainer
touch trainer/task.py
```

`cassava/` 目錄現在應該包含下列項目：

```
+ Dockerfile
+ trainer/
  + task.py
```

接著，開啟剛才建立的 `task.py` 檔案，然後複製下列程式碼。請將 `{your-gcs-bucket}` 改成您剛建立的 Cloud Storage 值區名稱。

```

import tensorflow as tf
import tensorflow_datasets as tfds
import os

PER_REPLICA_BATCH_SIZE = 64
EPOCHS = 2

# TODO: replace {your-gcs-bucket} with the name of the Storage bucket you created earlier
BUCKET = 'gs://{your-gcs-bucket}/mwms'

def preprocess_data(image, label):
    '''Resizes and scales images.'''

    image = tf.image.resize(image, (300,300))
    return tf.cast(image, tf.float32) / 255., label

def create_dataset(batch_size):
    '''Loads Cassava dataset and preprocesses data.'''

    data, info = tfds.load(name='cassava', as_supervised=True, with_info=True)
    number_of_classes = info.features['label'].num_classes
    train_data = data['train'].map(preprocess_data,
                                   num_parallel_calls=tf.data.experimental.AUTOTUNE)
    train_data = train_data.shuffle(1000)
    train_data = train_data.batch(batch_size)
    train_data = train_data.prefetch(tf.data.experimental.AUTOTUNE)

    # Set AutoShardPolicy
    options = tf.data.Options()
    options.experimental_distribute.auto_shard_policy =
tf.data.experimental.AutoShardPolicy.DATA
    train_data = train_data.with_options(options)

    return train_data, number_of_classes

def create_model(number_of_classes):
    '''Creates and compiles pretrained ResNet50 model.'''

    base_model = tf.keras.applications.ResNet50(weights='imagenet', include_top=False)
    x = base_model.output
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dense(1016, activation='relu')(x)
    predictions = tf.keras.layers.Dense(number_of_classes, activation='softmax')(x)
    model = tf.keras.Model(inputs=base_model.input, outputs=predictions)

    model.compile(
        loss='sparse_categorical_crossentropy',
        optimizer=tf.keras.optimizers.Adam(0.0001),
        metrics=['accuracy'])

```

```

return model

def _is_chief(task_type, task_id):
    '''Helper function. Determines if machine is chief.'''

    return task_type == 'chief'

def _get_temp_dir(dirpath, task_id):
    '''Helper function. Gets temporary directory for saving model.'''

    base_dirpath = 'workertemp_' + str(task_id)
    temp_dir = os.path.join(dirpath, base_dirpath)
    tf.io.gfile.makedirs(temp_dir)
    return temp_dir

def write_filepath(filepath, task_type, task_id):
    '''Helper function. Gets filepath to save model.'''

    dirpath = os.path.dirname(filepath)
    base = os.path.basename(filepath)
    if not _is_chief(task_type, task_id):
        dirpath = _get_temp_dir(dirpath, task_id)
    return os.path.join(dirpath, base)

def main():
    # Create strategy
    strategy = tf.distribute.MultiWorkerMirroredStrategy()

    # Get data
    global_batch_size = PER_REPLICA_BATCH_SIZE * strategy.num_replicas_in_sync
    train_data, number_of_classes = create_dataset(global_batch_size)

    # Wrap variable creation within strategy scope
    with strategy.scope():
        model = create_model(number_of_classes)

    model.fit(train_data, epochs=EPOCHS)

    # Determine type and task of the machine from
    # the strategy cluster resolver
    task_type, task_id = (strategy.cluster_resolver.task_type,
                          strategy.cluster_resolver.task_id)

    # Based on the type and task, write to the desired model path
    write_model_path = write_filepath(BUCKET, task_type, task_id)
    model.save(write_model_path)

```

```
if __name__ == "__main__":
    main()
```

建構容器之前，請先深入瞭解程式碼，其中使用了 `tf.distribute.Strategy` API 的 `MultiWorkerMirroredStrategy`。

`MultiWorkerMirroredStrategy` 專為在叢集中的多部機器之間分配訓練作業而設計，每部機器可能有多個 GPU。如要將訓練作業分配到單一裝置上的多個 GPU，請參閱 [MirroredStrategy](#)。

程式碼中有幾個元件是程式碼與 `MultiWorkerMirroredStrategy` 搭配運作時的必要元件。

1. 資料必須經過分片，也就是為每個工作人員指派整個資料集的一小部分。因此，在每個步驟中，每個 worker 都會處理不重疊的資料集元素，且全域批量大小相同。`tf.data.experimental.AutoShardPolicy` 會自動執行分片作業，可設為 `FILE` 或 `DATA`。在本例中，由於本著資料集並非以多個檔案的形式下載，因此 `create_dataset()` 函式會將 `AutoShardPolicy` 設為 `DATA`。不過，如果您未將政策設為 `DATA`，系統會啟動預設的 `AUTO` 政策，最終結果相同。如要進一步瞭解如何使用 `MultiWorkerMirroredStrategy` 進行資料集分片，請參閱[這篇文章](#)。
2. 在 `main()` 函式中，系統會建立 `MultiWorkerMirroredStrategy` 物件。接著，您要在策略範圍內包裝模型變數的建立作業。這項重要步驟會告知 TensorFlow 哪些變數應在副本之間鏡像處理。
3. 批次大小會依 `num_replicas_in_sync` 放大。這可確保每個副本在每個步驟中處理的範例數量相同。在 TensorFlow 中使用同步資料平行處理策略時，調整批量是最佳做法。
4. 在多個 worker 的情況下，儲存模型會稍微複雜，因為每個 worker 的目的地都必須不同。主要工作站會儲存到所需模型目錄，其他工作站則會將模型儲存到暫時目錄。為避免多個 worker 寫入相同位置，這些暫時目錄必須是專屬目錄。儲存作業可以包含集體作業，也就是所有 worker 都必須儲存，而不只是主 worker。`_is_chief()`、`_get_temp_dir()`、`write_filepath()` 函式和 `main()` 函式都包含樣板程式碼，有助於儲存模型。

請注意，如果您曾在其他環境中使用 `MultiWorkerMirroredStrategy`，可能已設定 `TF_CONFIG` 環境變數。Vertex AI 會自動為您設定 `TF_CONFIG`，因此您不需要在叢集中的每部機器上定義這個變數。

步驟 4：建構容器

在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

在 Google Container Registry 中，使用容器映像檔的 URI 定義變數：

```
IMAGE_URI="gcr.io/$PROJECT_ID/multiworker:cassava"
```

設定 Docker

```
gcloud auth configure-docker
```

接著，從 `cassava` 目錄的根層級執行下列指令，建構容器：

```
docker build ./ -t $IMAGE_URI
```

最後，將其推送至 Google Container Registry：

```
docker push $IMAGE_URI
```

容器推送至 Container Registry 後，您就可以啟動訓練工作。

步驟 6 · 約 35 分鐘

6. 在 Vertex AI 中執行多工作人員訓練工作

本實驗室使用 Google Container Registry 上的自訂容器進行自訂訓練，但您也可以使用預先建構的容器執行訓練工作。

首先，請前往 Cloud 控制台的 Vertex 專區，然後點選「訓練」部分：



Vertex AI



Dashboard



Datasets



Features



Labeling tasks



Workbench



Pipelines



Training



Experiments



Models



Endpoints



Batch predictions



步驟 1：設定訓練工作

按一下「建立」，輸入訓練工作的參數。

- 在「Dataset」(資料集) 下方，選取「No managed dataset」(沒有代管資料集)
- 然後選取「Custom training (advanced)」(自訂訓練 (進階)) 做為訓練方法，並點按「Continue」(繼續)。
- 在「Model name」(模型名稱) 中輸入 `multiworker-cassava` (或您想為模型命名的任何名稱)
- 按一下 [Continue] (繼續)。

在「容器設定」步驟中，選取「自訂容器」：

Select a pre-built container or build a custom container using ML frameworks (as well as non-ML dependencies, libraries and binaries) that are not otherwise supported.

[Learn more](#)



Pre-built container

View the list of [supported runtimes](#) including TensorFlow, scikit-learn and PyTorch versions



Custom container

Build a custom Docker container. Must be stored in [Container Registry](#)

在第一個方塊 (「容器映像檔」) 中，輸入上一節的 `IMAGE_URI` 變數值。應為 `gcr.io/your-cloud-project/multiworker:cassava`，並使用您自己的專案 ID。其餘欄位保留空白，然後點選「繼續」。

再次點選「繼續」，略過「超參數」步驟。

步驟 2：設定運算叢集

Vertex AI 提供 4 個工作人員集區，可涵蓋不同類型的機器工作。

工作站集區 0 會設定主要、主要、排程器或「主要」執行個體。在 `MultiWorkerMirroredStrategy` 中，所有機器都會指定為工作站，也就是執行複製運算的實體機器。除了每部機器都是工作站之外，還需要一個工作站來處理一些額外工作，例如儲存檢查點，以及將摘要檔案寫入 TensorBoard。這部機器稱為「首長」。主工作站一律只有一個，因此工作站集區 0 的工作站數量一律為 1。

在「Compute and pricing」（運算和價格）中，保留選取的區域，並將「Worker pool 0」（工作站集區 0）設定如下：

您可以選擇新增 GPU，但請注意，叢集中的每部機器應具有相同數量的 GPU。 `MultiWorkerMirroredStrategy` 新增 GPU 也會增加訓練工作的費用。

Worker pool 0

Machine type *

n1-standard-8, 8 vCPUs, 30 GiB memory

Accelerator type

Accelerators can speed up model training that involves intensive compute tasks. [Learn more](#)

Worker count

1

Disk type

SSD

Disk size (GB)

100

工作站集區 1 是設定叢集工作站的位置。

按照下列方式設定「工作站集區 1」：

Worker pool 1

Select the type of virtual machine to use for your training job's **second worker pool** nodes, as well as the number of worker replicas to use.

Machine type
n1-standard-8, 8 vCPUs, 30 GiB memory ▼

Accelerator type ▼

Worker count
1

Disk type
SSD ▼

Disk size (GB)
100

叢集現在已設定為有兩部僅含 CPU 的機器。執行訓練應用程式程式碼時，`MultiWorkerMirroredStrategy` 會將訓練作業分配到兩部機器上。

`MultiWorkerMirroredStrategy` 只有主要和工作者工作類型，因此不需要設定額外的工作者集區。不過，如果您要使用 TensorFlow 的 `ParameterServerStrategy`，則會在**工作站集區 2** 中設定參數伺服器。如要將評估人員新增至叢集，請在**工作站集區 3** 中設定該機器。

按一下「開始訓練」，啟動超參數調整工作。在控制台的「訓練」部分，您會看到「訓練管道」分頁下方顯示新啟動的工作：

Training

 [CREATE](#)

TRAINING PIPELINES

CUSTOM JOBS

HYPERPARAMETER TUNING JOBS

Training pipelines are the primary model training workflow in Vertex AI. You can use training pipelines to create an AutoML-trained model or a custom-trained model. For custom-trained models, training pipelines orchestrate custom training jobs and hyperparameter tuning with additional steps like adding a data set or uploading the model to Vertex AI for prediction serving. [Learn more](#)

Region

us-central1 (Iowa)



 Filter

Enter a property name

Name	ID	Job type	Model type
 multiworker-cassava	2119053575840595968	Training pipeline	 Custom

訓練工作大約需要 30 到 35 分鐘才能完成。

 恭喜！

您已瞭解如何使用 Vertex AI 執行下列作業：

- 啟動多工作站訓練工作，訓練自訂容器中提供的訓練程式碼。本範例使用 TensorFlow 模型，但您可以使用自訂或內建容器，訓練以任何架構建構的模型。

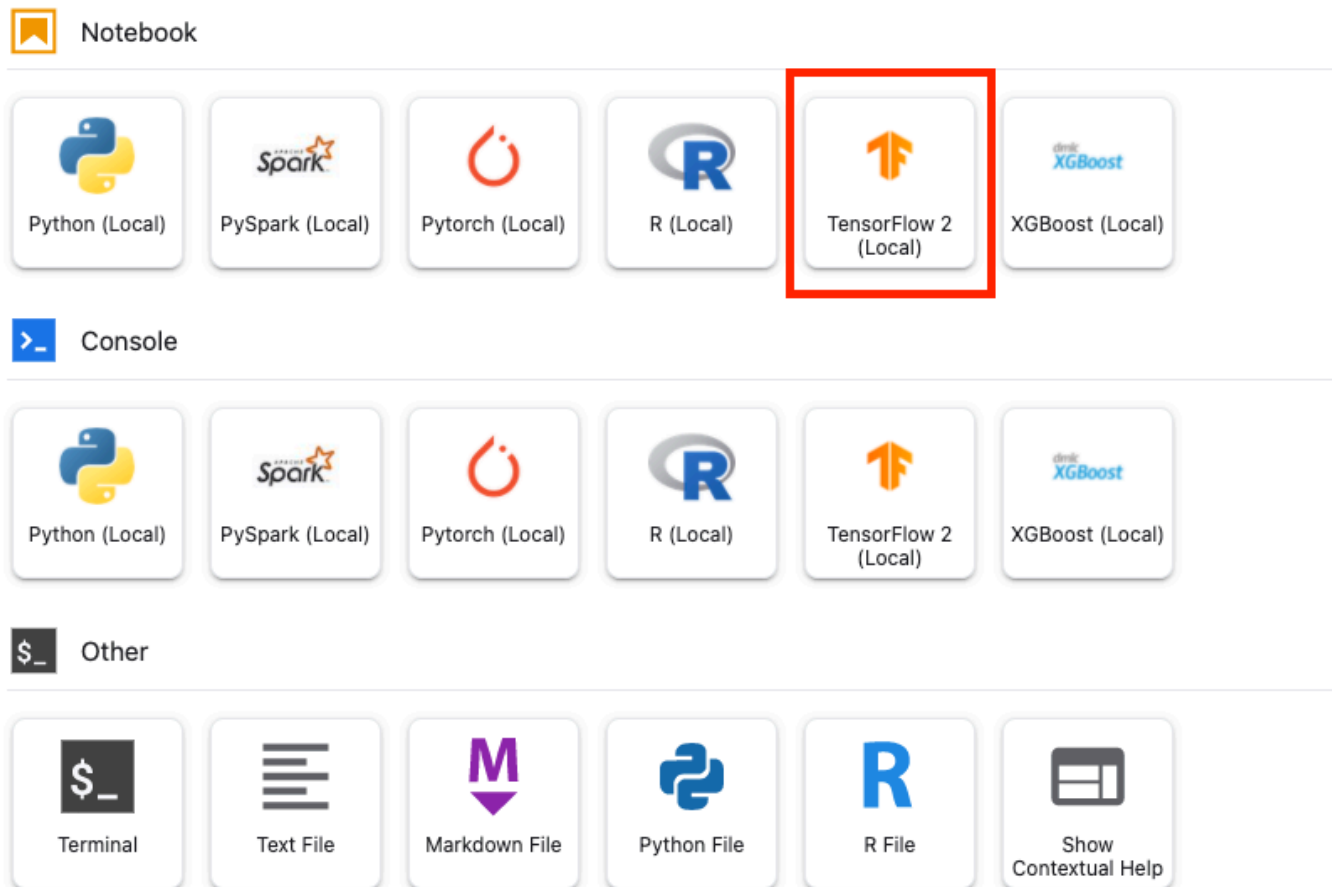
如要進一步瞭解 Vertex 的其他部分，請參閱[說明文件](#)。

步驟 7 · 約 35 分鐘

7. [選用] 使用 Vertex SDK

上一節說明如何透過 UI 啟動訓練工作。在本節中，您將瞭解如何使用 Vertex Python API 提交訓練工作。

返回筆記本執行個體，然後從啟動器建立 TensorFlow 2 筆記本：



匯入 Vertex AI SDK。

```
from google.cloud import aiplatform
```

如要啟動多 worker 訓練工作，您必須先定義 worker 集區規格。請注意，規格中的 GPU 完全是選用項目，如要使用僅含 CPU 的叢集，可以移除 `accelerator_type` 和 `accelerator_count`，如上一節所示。

```
# The spec of the worker pools including machine type and Docker image
# Be sure to replace {YOUR-PROJECT-ID} with your project ID.
worker_pool_specs=[
    {
        "replica_count": 1,
        "machine_spec": {
            "machine_type": "n1-standard-8", "accelerator_type": "NVIDIA_TESLA_V100",
"accelerator_count": 1
        },
        "container_spec": {"image_uri": "gcr.io/{YOUR-PROJECT-ID}/multiworker:cassava"}
    },
    {
        "replica_count": 1,
        "machine_spec": {
            "machine_type": "n1-standard-8", "accelerator_type": "NVIDIA_TESLA_V100",
"accelerator_count": 1
        },
        "container_spec": {"image_uri": "gcr.io/{YOUR-PROJECT-ID}/multiworker:cassava"}
    }
]
```

接著，請建立並執行 `CustomJob`。您必須將 `{YOUR_BUCKET}` 換成專案中的 bucket，做為暫存空間。你可以使用先前建立的同一個值區。

```
# Replace YOUR_BUCKET
my_multiworker_job = aiplatform.CustomJob(display_name='multiworker-cassava-sdk',
                                           worker_pool_specs=worker_pool_specs,
                                           staging_bucket='gs://{YOUR_BUCKET}')

my_multiworker_job.run()
```

在控制台的「訓練」部分，點選「自訂工作」分頁，即可查看訓練工作：

Training

 CREATE

TRAINING PIPELINES CUSTOM JOBS HYPERPARAMETER TUNING JOBS

Custom jobs specify how Vertex AI runs your custom training code, including worker pools, machine types and settings related to your Python training application and custom container. Custom jobs are only used by custom-trained models and not AutoML models.
[Learn more](#)

Region

us-central1 (Iowa)  

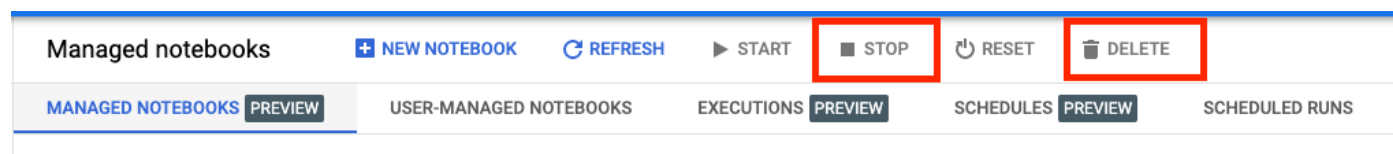
 Filter Enter a property name

Name	ID	Job type
 multiworker-cassava	2589679736900812800	Custom job

步驟 8 · 約 1 分鐘

8. 清除所用資源

由於我們將筆記本設定為在閒置 60 分鐘後逾時，因此不必擔心執行個體會關閉。如要手動關閉執行個體，請按一下控制台 Vertex AI Workbench 專區的「停止」按鈕。如要徹底刪除筆記本，請按一下「刪除」按鈕。



如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：

